

Difference between JPA, Hibernate and Spring Data JPA

AIM

To learn and study the difference between Java Persistence API (JPA), Hibernate and Spring Data JPA. See how Spring Data JPA helps to reduce the database operations compared to Hibernate.

THEORY

Java Persistence API (JPA)

The Java Persistence API (JPA) is a Java specification for mapping Java objects to relational database tables. It offers a standard set of interfaces and annotations to perform CRUD (Create, Read, Update, Delete) operations.

JPA itself is not an implementation. Instead, it defines rules that persistence providers have to follow.

Features of JPA

- Standard specification for Object Relational Mapping (ORM)
 - Uses annotations such as @Entity, @Table, @Id, and @Column
 - Database-independent API.
 - Supports JPQL (Java Persistence Query Language)
 - Requires an implementation such as Hibernate.
-

HIBERNATE

Hibernate is one of the most popular ORM frameworks and it is an implementation of the JPA specification.

Hibernate maps the Java objects directly with the database tables and automatically manages the database operations.

In contrast to Plain JDBC, developers need not write SQL queries for each and every operation.

Features of HIBERNATE

- Implements JPA
- Supports automatic table mapping
- Provides Session and SessionFactory
- Handles transactions

- Supports caching for improved performance
- Reduces SQL coding

In Hibernate, developers manually manage:

- Opening sessions
- Beginning transactions
- Committing transactions
- Rolling back transactions
- Closing sessions

SPRING DATA JPA

Spring Data JPA is a module in the Spring framework that builds on JPA. It does not implement JPA directly. Instead, it uses an existing implementation like Hibernate while greatly cutting down on boilerplate code. Developers primarily work with repository interfaces rather than writing implementation classes.

Features of SPRING DATA JPA

- Uses repository interfaces
- Automatically generates CRUD methods
- Reduces boilerplate code
- Integrates easily with Spring Boot
- Automatically manages transactions using annotations

- Improves code readability and maintainability

COMPARISON of JPA, HIBERNATE and SPRING DATA JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Spring Module
Implementation	No	Yes	No
Purpose	Defines persistence standard	Implements JPA	Simplifies JPA usage
Boilerplate Code	Moderate	High	Very Low
Transaction Management	Through implementation	Manual	Automatic using @Transactional
CRUD Operations	Defined by specification	Implemented manually	Repository methods
Database Mapping	Supported	Supported	Supported through JPA
Ease of Development	Medium	Medium	High

CODE ANALYSIS

Hibernate Implementations

```
public Integer addEmployee(Employee employee){  
  
    Session session = factory.openSession();  
  
    Transaction tx = null;  
  
    Integer employeeID = null;  
  
    try {  
  
        tx = session.beginTransaction();  
  
        employeeID = (Integer) session.save(employee);  
  
        tx.commit();  
    } catch (HibernateException e) {  
  
        if (tx != null)  
            tx.rollback();  
  
        e.printStackTrace();  
    } finally {  
  
        session.close();  
  
    }  
  
    return employeeID;  
}
```

Observation

Manual actions to perform by the developer are:

Open the session

Begin the transaction

Persist the object

Commit the transaction

Process exceptions

Rollback if needed

Close the session

The developer is burdened with additional coding and transaction management complexity because of this.

SPRING DATA JPA IMPLEMENTATION REPOSITORY

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {}
```

SERVICES

```
@Autowired

private EmployeeRepository employeeRepository;


@Transactional

public void addEmployee(Employee employee) {

    employeeRepository.save(employee);

}
```

Observation

Repository method implementations for `save()`, `findAll()`, `findById()`, `delete()`, and `count()` are automatically provided to you by Spring Data JPA and you will not have to write any SQL or manage sessions yourself, just call these methods.

Handling transactions is also automatically performed with the help of the `@Transactional` annotation.

CODE DIFFERENCES

HIBERNATE

- Requires SessionFactory
- Requires Session

- Manual transaction handling
- More boilerplate code
- Greater control over persistence

JAVA DATA JPA

- Uses JpaRepository
- Automatic CRUD implementation
- Automatic transaction management
- Less code
- Easier to maintain

CONCLUSION

In this instance, the investigation into JPA, Hibernate and Spring Data JPA provided sufficient data to distinguish how each works independently. Spring Data JPA makes performing operations against the database easier than either of the other two due to being able to eliminate code duplication (reducing boilerplate) and allowing the system to perform transaction management automatically.

Our comparison of code samples indicates that Spring Data JPA produces less clutter in terms of code and is easier for developers to understand and maintain, while using Hibernate as its JPA implementation source.